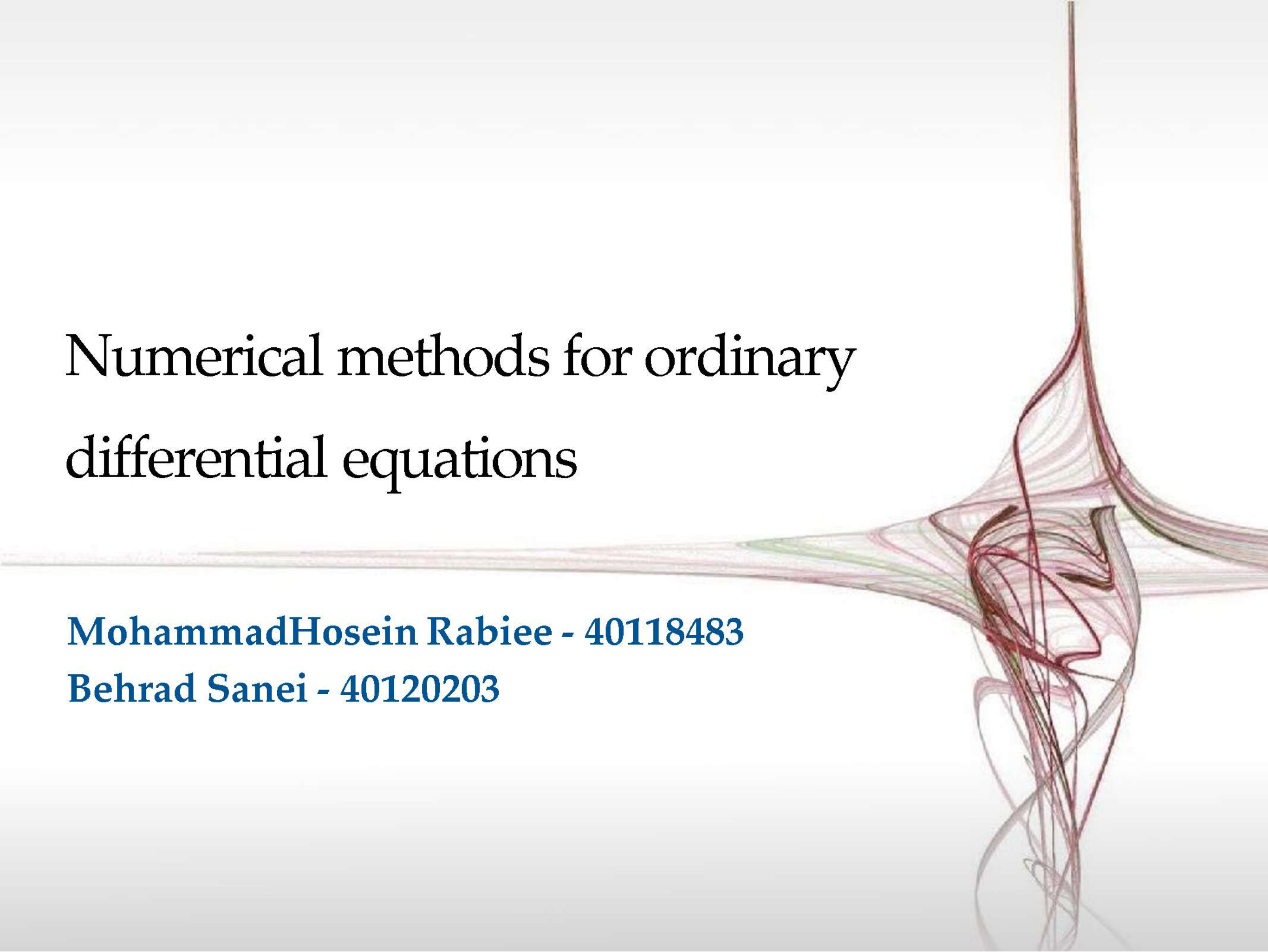


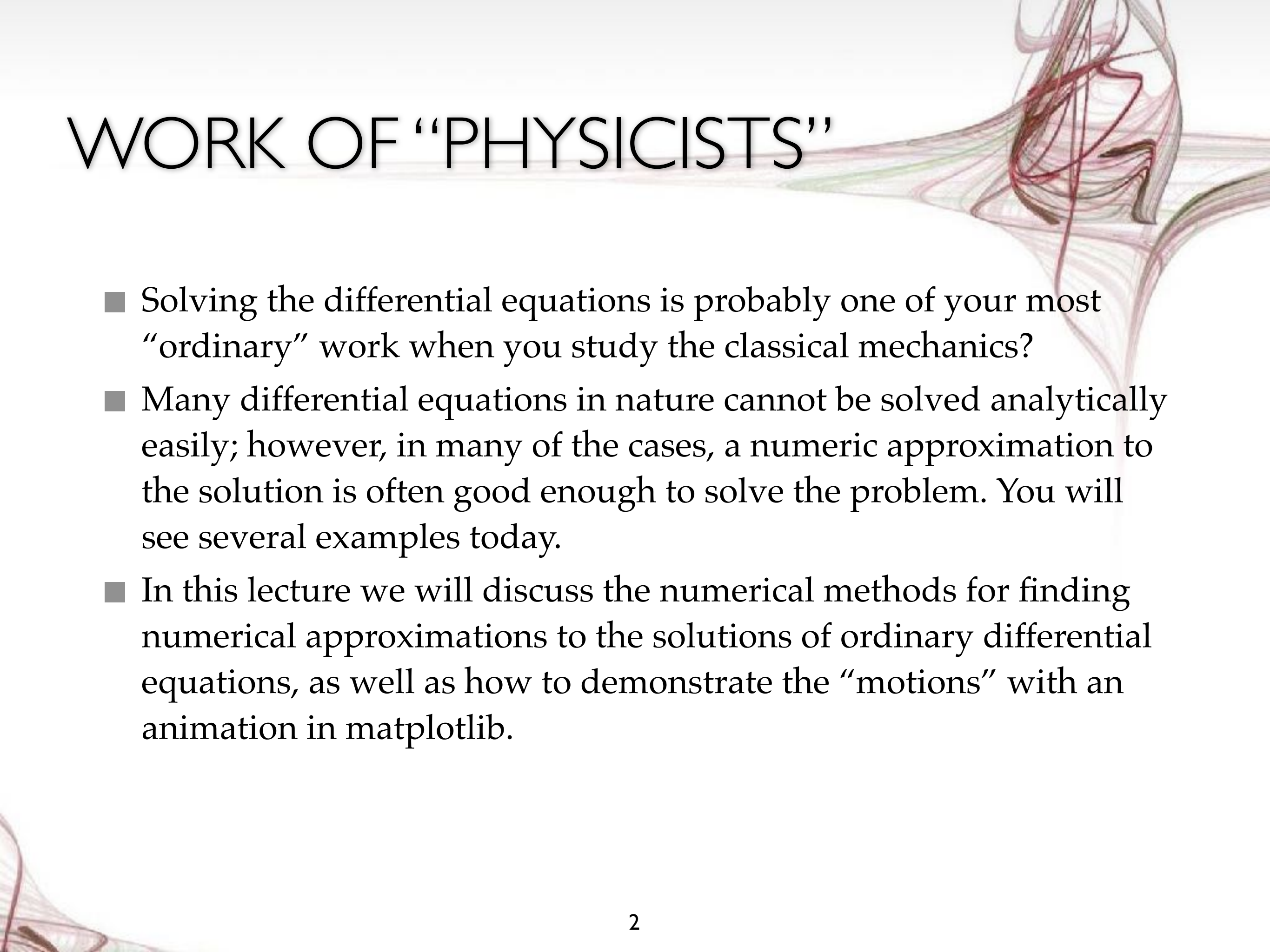
Numerical methods for ordinary differential equations

MohammadHosein Rabiee - 40118483

Behrad Sanei - 40120203



WORK OF “PHYSICISTS”



- Solving the differential equations is probably one of your most “ordinary” work when you study the classical mechanics?
- Many differential equations in nature cannot be solved analytically easily; however, in many of the cases, a numeric approximation to the solution is often good enough to solve the problem. You will see several examples today.
- In this lecture we will discuss the numerical methods for finding numerical approximations to the solutions of ordinary differential equations, as well as how to demonstrate the “motions” with an animation in matplotlib.

THE BASIS: A BRAINLESS EXAMPLE

- Let's try to solve such a (mostly) trivial differential equation:

$$\frac{dy}{dt} = f(y, t) = y \quad \text{with the initial condition: } t = 0, y = 1$$

- You should know the obvious solution is — $y = \exp(t)$

$$\frac{dy}{dt} = f(y, t) \quad \text{Actually, this is the **general form** of any first-order ordinary differential equation.}$$

In general, it can be very complicated, but it's still a 1st order ODE, e.g.

$$\frac{dy}{dt} = f(y, t) = y^3 \cdot t^2 + \sin(t + y) + \sqrt{t + y}$$

THE NUMERICAL SOLUTION

- Here are the minimal algorithm — integrate the differential equation by **one step in t**:

$$\frac{dy}{dt} = f(y, t)$$

$$\frac{y(t_{n+1}) - y(t_n)}{h} = f(y, t_n) \quad \longrightarrow \quad y_{n+1} \approx y_n + h \cdot f(y_n, t_n)$$

next step

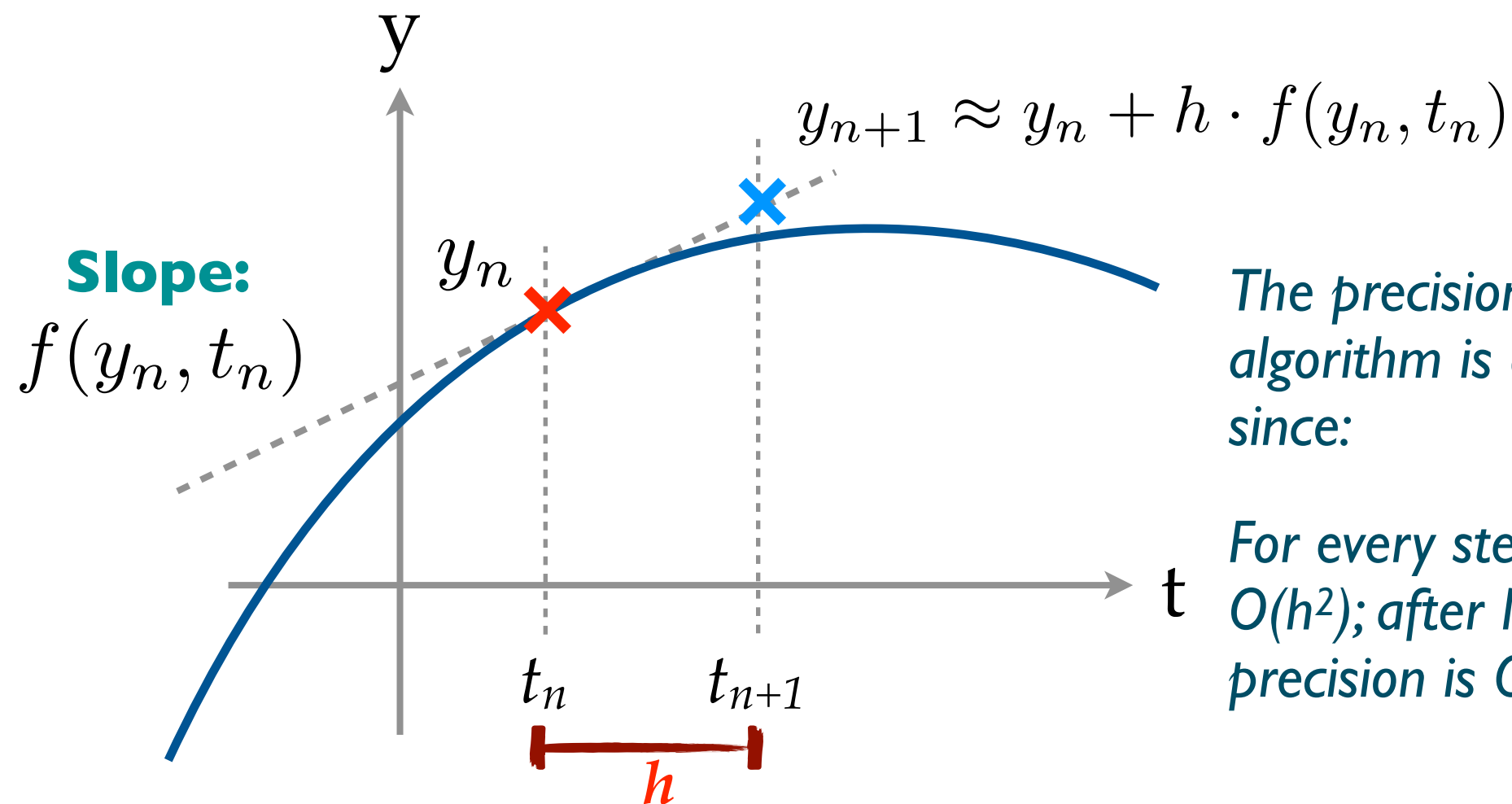
current step

For our trivial example: $\frac{dy}{dt} = y \quad \longrightarrow \quad y_{n+1} \approx y_n + h \cdot y_n$

This is the classical **Euler algorithm (method)**

EULER ALGORITHM

- A more graphical explanation is as like this:



The precision of this Euler algorithm is only up to $O(h)$ since:

For every step the precision is of $O(h^2)$; after $N \sim O(1/h)$ steps the precision is $O(h)$.

EULER ALGORITHM (II)

- Let's prepare a simple code to see how it works:

```
import math
def f(t,y): return y
t, y = 0., 1.  ⇐ Initial conditions (t = 0, y = 1)
h = 0.001  ⇐ stepping in t
while t<1.:
    k1 = f(t, y) ⇐ the given f(y,t) function
    y += h*k1
    t += h
y_exact = math.exp(t)
print('Euler method: %.16f, exact: %.16f, diff: %.16f' % \
(y,y_exact,abs(y-y_exact)))
```

l206-example-01.py

```
Euler method: 2.7169239322358960,
exact:         2.7182818284590469,
diff:         0.0013578962231509 ⇐ Indeed the precision is of  $O(h)$ 
```

SECOND ORDER RUNGE-KUTTA METHOD

- Surely one can introduce a similar trick of error reduction we have played though out the latter half of the semester.
- Here comes the **Runge-Kutta algorithm** for integrating differential equations, which is based on a formal integration:

$$\frac{dy}{dt} = f(y, t) \quad \longrightarrow \quad \begin{aligned} y(t) &= \int f(t, y) dt \\ y_{n+1} &= y_n + \int_{t_n}^{t_{n+1}} f(t, y) dt \end{aligned}$$

Expand **$f(t, y)$** in a Taylor series around $(t, y) = (t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}})$

$$f(t, y) = f(t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}}) + (t - t_{n+\frac{1}{2}}) \cdot \frac{df}{dt}(t_{n+\frac{1}{2}}) + O(h^2)$$

Something smells familiar?

SECOND ORDER RUNGE-KUTTA METHOD (II)

$$f(t, y) = f(t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}}) + (t - t_{n+\frac{1}{2}}) \cdot \frac{df}{dt}(t_{n+\frac{1}{2}}) + O(h^2)$$

Insert the expansion
into the integration:

$$\int_{t_n}^{t_{n+1}} f(t, y) dt = \int_{t_n}^{t_{n+1}} f(t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}}) dt + \int_{t_n}^{t_{n+1}} (t - t_{n+\frac{1}{2}}) \cdot \frac{df}{dt}(t_{n+\frac{1}{2}}) dt + \dots$$

It's just a number (slope)!

Linear (first order) term must be cancelled

Insert the integral back:

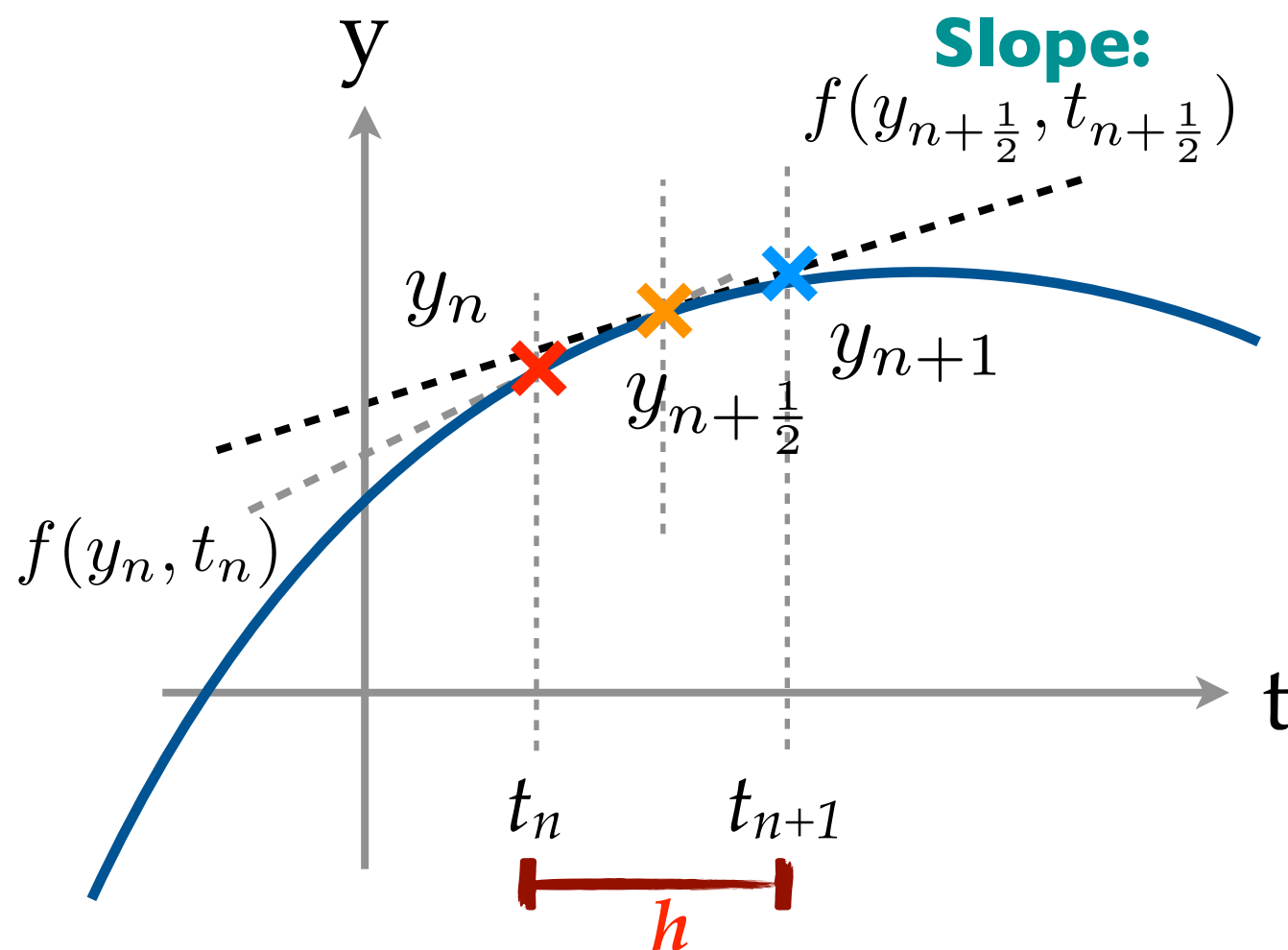
$$\int_{t_n}^{t_{n+1}} f(t, y) dt \approx h \cdot f(t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}})$$

$$\longrightarrow y_{n+1} \approx y_n + h \cdot f(t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}}) + O(h^3)$$

If one knows the solution **half-step in the future** — the $O(h^2)$ term can be cancelled. **BUT HOW?**

SECOND ORDER RUNGE-KUTTA METHOD (III)

- The trick: use the **Euler's method to solve half-step first**, starting from the given initial conditions:



$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n)$$

$$t_{n+\frac{1}{2}} = t_n + \frac{h}{2}$$

$$y_{n+1} \approx y_n + h \cdot f(t_{n+\frac{1}{2}}, y_{n+\frac{1}{2}})$$



Explicit formulae

$$k_1 = f(t_n, y_n)$$

$$k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_1)$$

$$y_{n+1} \approx y_n + h \cdot k_2 + O(h^3)$$

IMPLEMENTATION OF “RK2”

- The coding is actually extremely simple:

```
t, y = 0., 1.  
h = 0.001      ⇐ Initial conditions and stepping (t = 0, y = 1, h = 0.001)  
  
while t < 1.:  
    k1 = f(t, y)      ⇐ use Euler method to solve half-step  
    k2 = f(t+0.5*h, y+0.5*h*k1)  
    y += h*k2      ⇐ full step jump  
    t += h  
  
y_exact = math.exp(t)  
print('RK2 method: %.16f, exact: %.16f, diff: %.16f' % \  
      (y, y_exact, abs(y-y_exact)))
```

I206-example-02.py

RK2 method:	2.7182813757517628,
exact:	2.7182818284590469,
diff:	0.0000004527072841

For every step the precision is of $O(h^3)$; after N steps the precision is $O(h^2)$.

FOURTH ORDER RUNGE-KUTTA

- The **4th order Runge-Kutta method** provides an excellent balance of power, precision, and programming simplicity. Using a similar idea of the 2nd order version, one could have these formulae:

$$\left\{ \begin{array}{l} k_1 = f(t_n, y_n) \\ k_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_1\right) \\ k_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} \cdot k_2\right) \\ k_4 = f(t_n + h, y_n + h \cdot k_3) \end{array} \right.$$
$$y_{n+1} \approx y_n + \frac{h}{6} \cdot (k_1 + 2k_2 + 2k_3 + k_4) + O(h^5)$$

Basically the 4th order Runge-Kutta has a precision of $O(h^5)$ at each step, an over all **$O(h^4)$** precision.

Actually, the RK4 is a variation of **Simpson's method...**

IMPLEMENTATION OF “RK4”

- The RK4 routine is not too different from the previous RK2 code!

```
t, y = 0., 1.  ⇐ The same initial conditions & stepping
h = 0.001
while t < 1.:
    k1 = f(t, y)  RK4 solver
    k2 = f(t+0.5*h, y+0.5*h*k1)  ⇐ Simply calculate k1~k4 in a sequence
    k3 = f(t+0.5*h, y+0.5*h*k2)
    k4 = f(t+h, y+h*k3)
    y += h/6.*(k1+2.*k2+2.*k3+k4)  ⇐ Jump to the next step
    t += h
y_exact = math.exp(t)
print('RK4 method: %.16f, exact: %.16f, diff: %.16f' % \
      (y, y_exact, abs(y-y_exact)))
```

I206-example-03.py

RK4 method:	2.7182818284590247,
exact:	2.7182818284590469,
diff:	0.0000000000000000222 ⇐ Precision is of $O(h^4)$!

PRECISION EVOLUTION

- Let's write a small code to demonstrate the “precision” of the solution as it evolves.
- You should be able to see the “accumulation” of numerical errors.

```
vt,y1,vy2,vy4 = [],[],[],[] ⇐ List for storing  
the output  
t = 0.  
y1 = y2 = y4 = 1.  
h = 0.001  
while t<200.: ⇐ now we calculate up to t=200
```

```
k1 = f(t, y1) Euler method  
y1 += h*k1
```

```
k1 = f(t, y2) RK2  
k2 = f(t+0.5*h, y2+0.5*h*k1)  
y2 += h*k2
```

```
k1 = f(t, y4) RK4  
k2 = f(t+0.5*h, y4+0.5*h*k1)  
k3 = f(t+0.5*h, y4+0.5*h*k2)  
k4 = f(t+h, y4+h*k3)  
y4 += h/6.*(k1+2.*k2+2.*k3+k4)
```

```
t += h
```

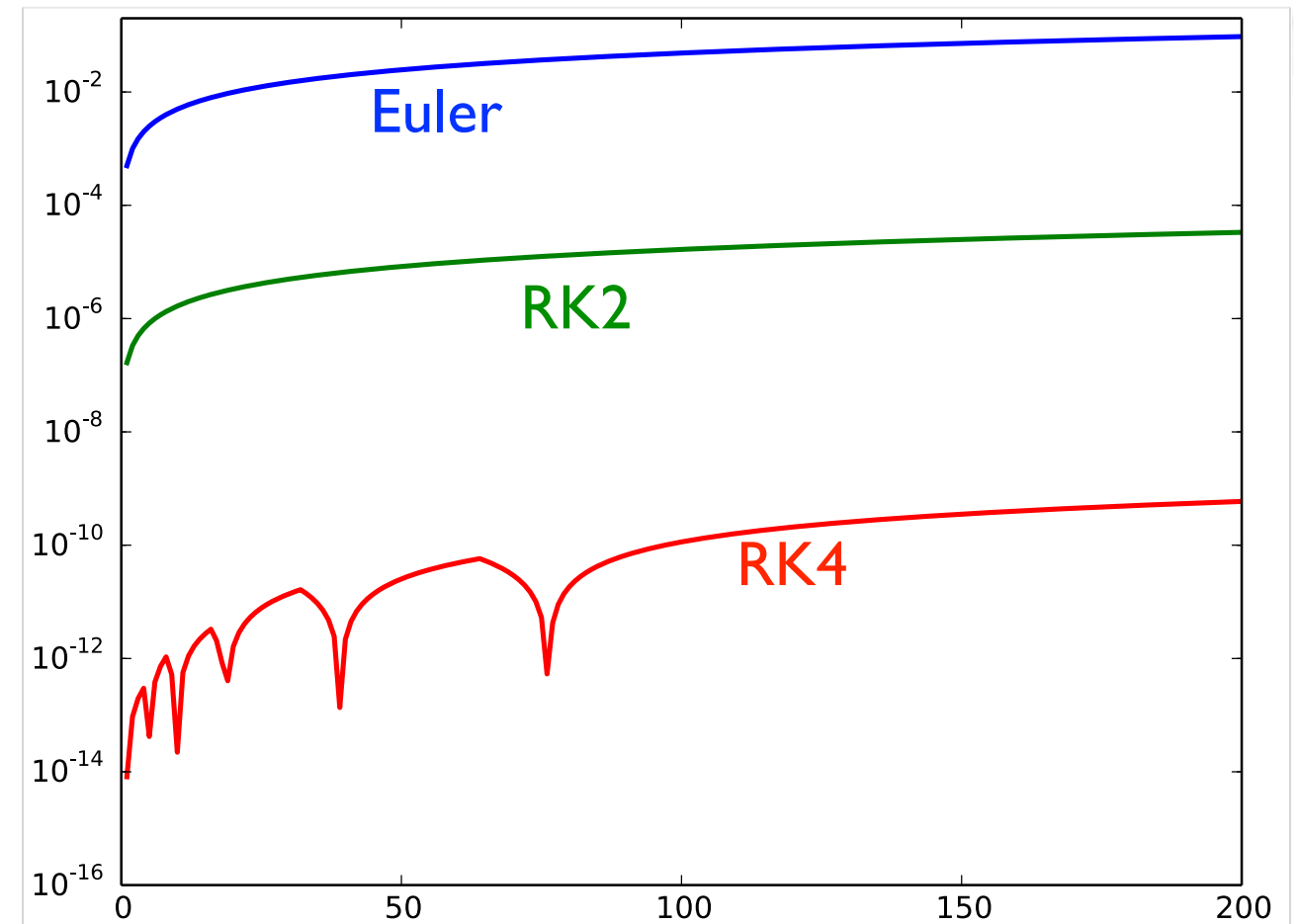
```
vt.append(t)  
vy1.append(abs(y1-np.exp(t))/np.exp(t))  
vy2.append(abs(y2-np.exp(t))/np.exp(t))  
vy4.append(abs(y4-np.exp(t))/np.exp(t))
```

↑↑ Store the relative errors

I206-example-03a.py (partial)

PRECISION EVOLUTION (II)

- Just make a simple plot.
- The initial uncertainties are of $O(h)$, $O(h^2)$, and $O(h^4)$.
- After 200,000 steps or more, the accumulated errors can be large.



```
plt.plot(vt, vy1, lw=2, c='Blue')
plt.plot(vt, vy2, lw=2, c='Green')
plt.plot(vt, vy4, lw=2, c='Red')
plt.yscale('log')
plt.ylim(1E-16, 0.2)
plt.show()
```

↑↑ Draw the relative differences

I206-example-03a.py (partial)